ECEN 248 - Lab Report

Lab #3

Rudimentary Adder Circuits

Section: 524

Ramya Kannan

UIN: 635002011

10/02/2025

TA: Reed Clardy

Objectives:
The purpose of this lab is to develop and design a half adder, full adder, and ripply carry adder. This lab introduces concepts on the design of simple combinational adder circuits, simple arithmetic logic, and Karnaugh Maps to create simplified logic expressions. We will continue to implement the DC Power supply and breadboard to test the adder circuits. This lab will provide hands-on experience implementing digital circuits using integrated circuits, and further our understanding of half, full, and ripply carry adders.

Design:
First, we set up the power supply for the experiment. While the power supply is turned off, we connected the power supply leads to the breadboard using only channel one (yellow): red cable connected to the power supply's positive terminal and the pin of this cable to the positive power rail on the breadboard, and the black cable connected to the power supply's negative terminal and the pin of this cable to the negative power rail.

Next, we set up the breadboard with 3 gates across the partition: AND gate, OR gate, and XOR gate. We connected each gate to the corresponding VCC and GRND lines. Make sure the notch of each gate is on the top side. Next, we set up our DIP switches, but instead, we use jumper wires to imitate their use. For each input (for the half adder) A_0 and B_0, we placed the jumper wire on one side connected to the negative rail (for now), and the other on the circuit area of the breadboard (for each input, it has a corresponding row on the board where the other side of the jumper wire is connected).

First, for the half adder A_0 and B_0 inputs were carried from the DIP switches to the XOR gate. The **output of the XOR gate is S (sum)**. A_0 and B_0 are the inputs for the first AND gate, **outputting the Cout.** Since we are going to be reading our outputs S_0 (Sum) and Cout_0 (Carry out) from the LED, we have to set these up. We connected one of the pins on the LED to the negative power rail (acting as a GRND) and then the other to the row on the row of where it's going to read the output from.

Second, the full adder is built off the half adder. The output from the previous XOR gate became the first input of the second AND gate, and the second was the Cin input. The output from this AND goes into the first OR gate as one of the inputs, the second input for this OR gate is the output from the first AND gate (A_0 + B_0). The output of **this OR gate is the Cout_0, which is represented by an LED.** Then the output from the first XOR gate (A XOR B) is the first input into the second XOR gate, and the second input for this is C_in. The output of the **second XOR gate is S_0 (sum)., which is represented by an LED.**

Lastly, the ripply carry adder is built off the full adder. The output from the first OR gate (from the full adder, outputting Cout_0) is the first input for the third XOR gate. The second input for the third XOR gate is the output of a fourth XOR gate whose inputs are A_1 and B_1. The output of the **third XOR gate is S_1 (sum), which is represented by an LED.** Next the output from the first OR gate (from the full adder, outputting Cout_0) is the first input in the third AND gate, and the second input is the output of the

fourth XOR gate (inputs are A_1 and B_1). This output is fed into the last OR gate, and the second input for this OR can is the output of another AND gate whose impetus are A_1 and B_1. The output of the **last OR gate is Cout_2, which is represented by an LED**

When testing each combination for the half adder, full adder, and the ripple carry adder, make sure you turn off the DC Power supply when changing the inputs from HIGH to LOW.

Results:

The half adder is the implementation of one XOR gate for the Sum and one AND gate for the carry out (Cout). Both outputs were displayed through LEDs. We tested all input combinations, and the behavior matched the theoretical truth table for all cases. When both A and B were 0, no LEDS were lit up. When A = 1 and B = 0, or A = 0 and B = 1, only the Sum LED lit up. Lastly, when both A and B were 1, the Cout (Carryout) LED lit up. These tests for the half adder had to be confirmed to continue to the full adder.

Secondly, the full adder was constructed with one OR gate, two XOR gates, and two AND gates. Some examples include: A =1, B = 1, Cin = 1, the outputs Sum = 1 and Cout = 1. Another example is when A = 0, B = 1, and Cin = 1, the outputs Sum = 0, and Cout = 1. Given a partially completed theoretical truth table for the full adder, we used that to test and verify the outputs of the full adder circuit we created. The addition results ranged from 0 to 3.

Lastly, the 2-bit ripple carry adder was built with two full adders (built from half to full to ripple carry). There were 3 different outputs: S0, S1, and Cout. Each of the outputs was displayed through LEDs. The circuit was successful for many test cases and matched the expected behavior. We tested additional input combinations and confirmed that carry propagation between two adders was successful. An example of an input combination we tested was: A0 = 1, A1 = 1, B0 = 0, B1 = 0, Cin = 0. The outputs were Cout = 0, S1 = 0 and S0 = 1. The LEDs provided the visual results, which made testing the half adder, full adder, and ripple carry adder super efficient.

To summarize the results, the circuits worked as expected. The LEDs proved to be accurate and efficient tools for demonstrating how carry signals propagate across stages in multi-bit addition, while also clearly indicating whether the designs matched the expected results. Beginning with a half adder and progressing through to the ripple carry adder with thorough testing and verification, this lab highlighted the critical importance of consistently validating circuits against truth tables at every stage of development.

Conclusion:

We successfully tested and designed the implementation of a half adder, full adder, and a ripple carry adder. We began with the half adder, then built off of that to build the full adder, and then built off of that to build the ripply carry adder. The pre-lab was useful for the lab because we created truth tables and schematics to allow verification of the experimental results during the testing of the lab. We also demonstrated how to translate the output of logical expressions into physics circuits using integrated circuits and LEDs.

Post-Lab Deliverables:

1. **Provide all design items found in the pre-lab deliverables. If you found that a design needed corrections while executing the lab, supply the updated version of that material.**

Table 1. Half Adder Truth Table

A	**B**	S_0	$Cout_0$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

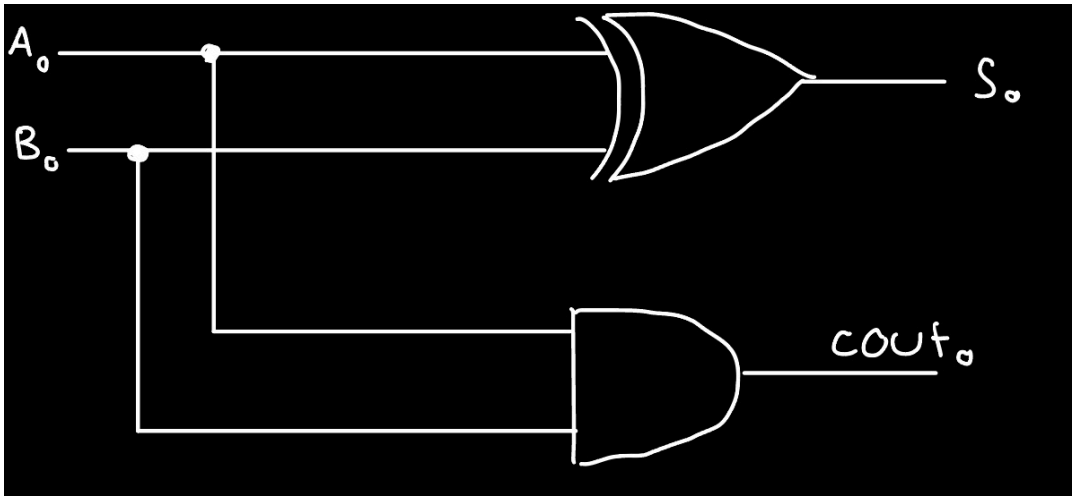
Table 2. Half Adder S(Sum) K-Map

B A	0	1
`0	0	1
1	1	0

Table 3. Cout (Carry out) K-Map for Half Adder

B A	0	1
`0	0	0
1	0	1

Figure 1. Gate-Level Half Adder Schematic



Logic Expressions:
From the K-maps, boolean expressions can be derived for the half adder outputs. It can be implemented with XOR gates. Using XOR gates reduces the total gate count from 5 gates to 2 gates, resulting in a simplified and faster operation.

Without XOR Gates

S = A'B + AB'
Cout = AB

With XOR Gates

S = A ⊕ B
Cout = AB

Table 4. Truth Table for Full Adder

A	B	C_{in}	S_0	$Cout_0$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

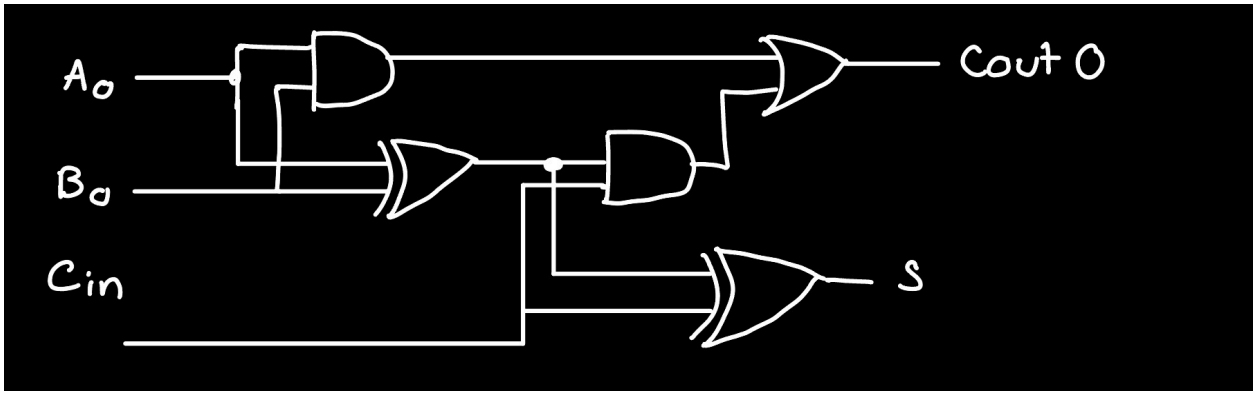
Table 5: Full Adder Sum (S) K-Map

BC A	**00**	**01**	**11**	**10**
0	0	1	0	1
1	1	0	1	0

Table 6: Full Adder Cout (Carryout) K-Map

BC A	**00**	**01**	**11**	**10**
0	0	0	1	0
1	0	1	1	1

Figure 2. Gate-Level Full Adder Schematic



Logic Expressions:

From the K-maps, boolean expressions can be derived for the full adder outputs. It can be implemented with XOR gates. Using XOR gates reduces the complexity of the sum expressions from four separate minterms to a single three-input XOR gate. The carry expression is also optimized by reusing the intermediate XOR result.

With XOR Gates:

$S = A \oplus B \oplus Cin$

$Cout = AB + Cin(A \oplus B)$

Without XOR Gates:

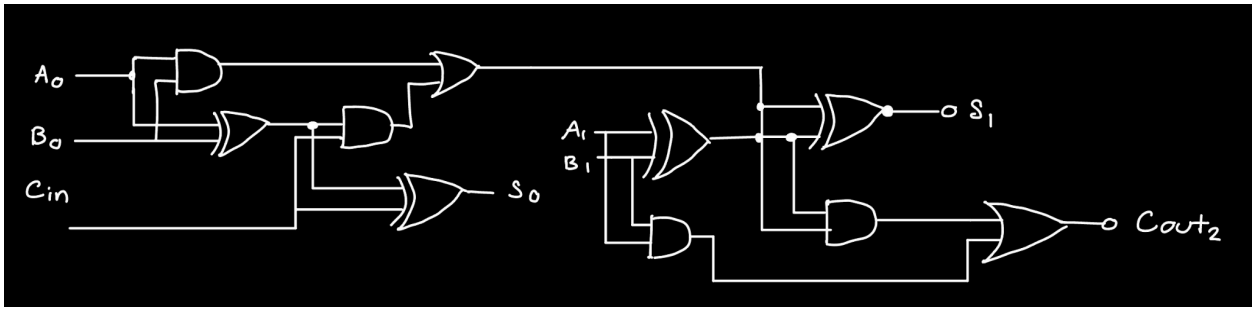
S = A’B’Cin + A’BC’ + AB’Cin’ + ABCin
Cout = AB + ACin + BCin

Table 7. Truth Table for Ripple Carry Adder

A_1	A_0	B_1	B_0	C_{in}	C_{out}	S_1	S_0
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	0	1	0
0	0	1	0	0	0	1	0
0	0	1	0	1	0	1	1
0	0	1	1	0	0	1	1
0	0	1	1	1	1	0	0
0	1	0	0	0	0	0	1
0	1	0	0	1	0	1	0
0	1	0	1	0	0	1	0
0	1	0	1	1	0	1	1
0	1	1	0	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	0	1	0	0
1	0	1	0	1	1	0	1
1	0	1	1	0	1	0	1

1	0	1	1	1	1	1	0
1	1	0	0	0	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	0	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	0	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1

Figure 3. Gate-Level Ripple Carry Adder Schematic



2. Determine the worst case propagation delay for your full adder design. Assume each gate has the same delay of 1 unit. Show the maximum delay path in your schematic. The maximum delay path is known as the critical path for that particular combinational block

The worst case propagation delay for the fuller adder design is both B_0 or A_0. For both it has a maximum delay of 3 units, making both of them critical paths. Here is the schematic with both critical paths highlighted.

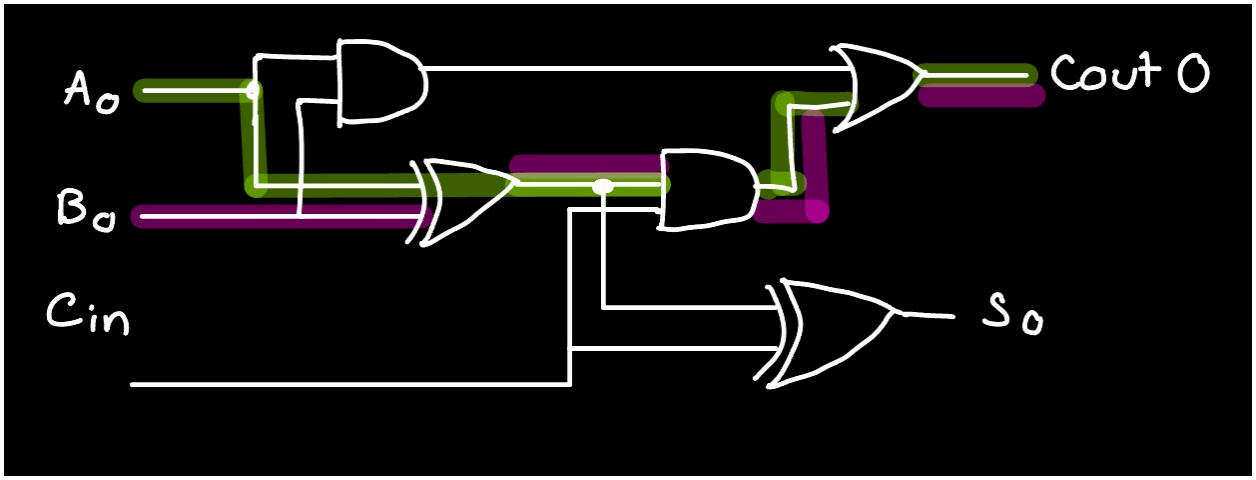


Figure 4. Circuit diagrams with the critical paths highlighted (equivalent maximum delays)

3. Design a 2-bit carry ripple adder assuming you only have half adder circuits and OR gates to work with. Draw up a schematic for your design using half adder building blocks and OR gates. Be sure the clearly label all inputs and outputs of your blocks

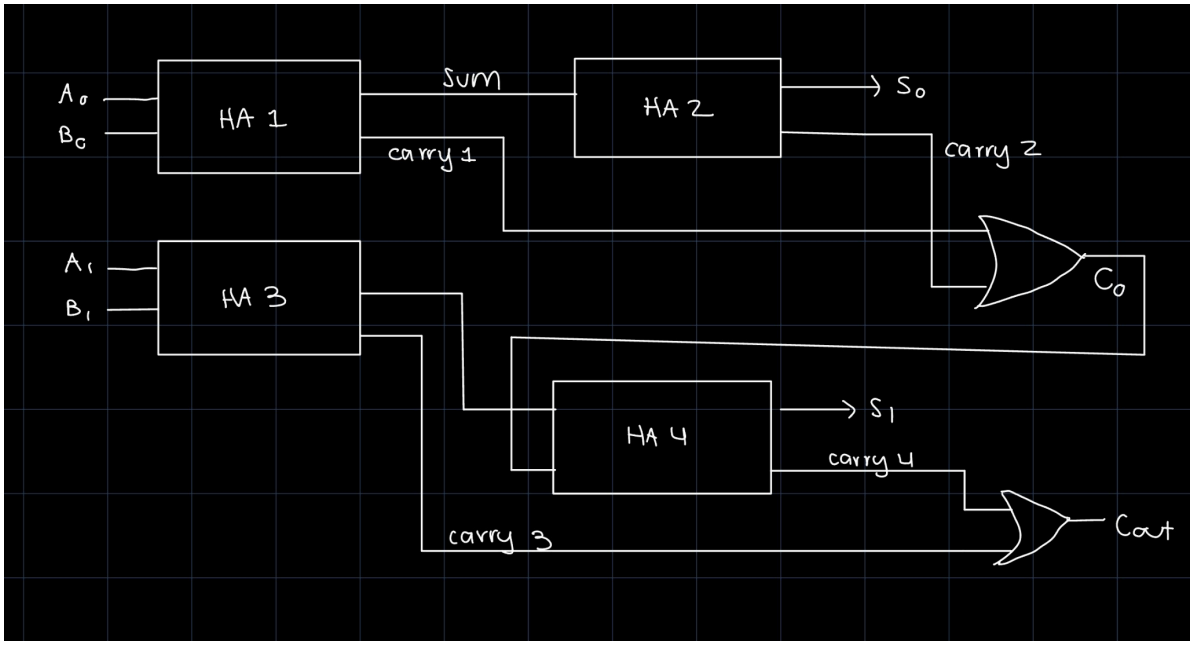


Figure 5. 2-bit carry ripple adder only using half adders and OR gates

Student Feedback:

1. What did you like most about the lab assignment and why? What did you like least about it and why?

What I liked most about the lab was that the circuits builded off each other. Like the half adder to full adder to ripple carry adder. What I liked least about this lab was that my wires were breaking a lot. .

2. Were there any section of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?
No, there weren't any sections that were unclear in the lab. It was pretty clear and the TA's were helpful if we had any questions.

3. What suggestions do you have to improve the overall lab assignment?
I have no suggestion to improve the overall lab assignment.